

Software Engineering Design patterns

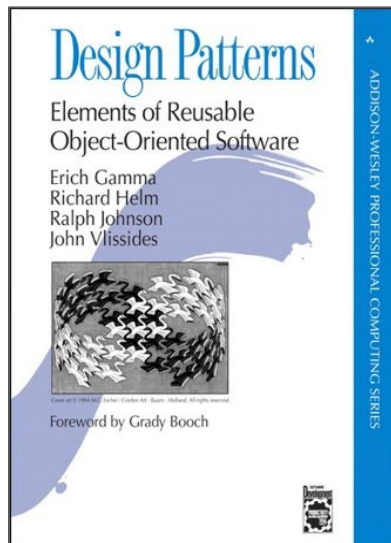
Credit: Slides adapted from Marco Tulio Valente's book Modern Software Engineering.

Plan

- Design patterns
- Factory
- Singleton
- Proxy
- Adapter
- Facade
- Decorator
- Strategy
- Observer
- Template

Design Patterns

- Common solutions to recurring design problems faced by developers



Authors presented a catalog of solutions for recurring problems in software design.

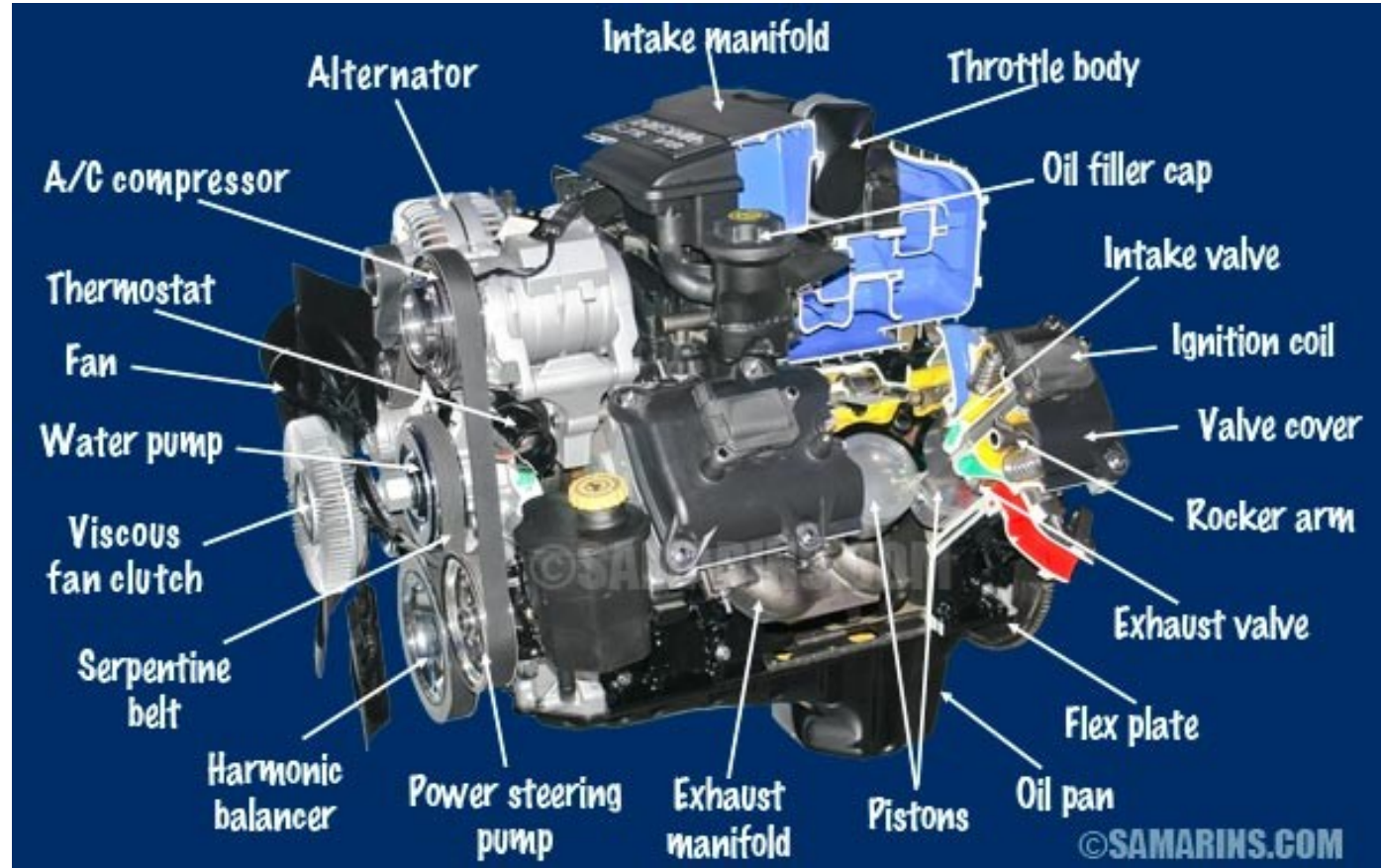
They called the solutions in the book design patterns and defined them as follows:

Design patterns are descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context.

Usage #1: Design Reuse

- When facing a design problem, we may reuse an existing design pattern to avoid reinventing the wheel

What components constitute a car engine?



Similarly, which components (or patterns) can be reused in software design?

- e.g. to ensure only one instance of a class exists (e.g., a configuration manager) we may use singleton,
- e.g. in any system requiring notification mechanisms, we can use the observer patterns ...etc.

Usage #2: Vocabulary

- Design patterns provide a shared vocabulary for discussions, documentation, and communication

```
public abstract class DocumentBuilderFactory  
extends Object
```

Defines a **factory API** that enables applications to obtain a parser that produces DOM object trees from XML documents.

Documentation for a class in the Java API

Developers can also benefit from knowing design patterns in two scenarios:

- When implementing their own code. In this case, knowledge of design patterns can help them apply a tested and validated design solution in their implementation. (DPs aim to create flexible and extensible designs)
- When using third-party code, such as the `DocumentBuilderFactory` class in the figure. In this case, knowledge of design patterns can assist them in understanding the behavior and structure of the code they intend to reuse.

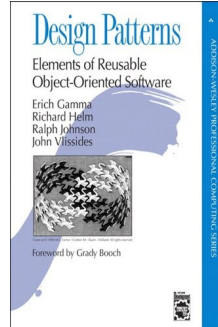
23 patterns

Creational	Structural	Behavioural
Abstract Factory Factory Method Singleton Builder Prototype	Proxy Adapter Facade Decorator Bridge Composite Flyweight	Strategy Observer Template Method Visitor Chain of Responsibility Command Interpreter Iterator Mediator Memento State

provide various object creation mechanisms, which increase flexibility and reuse of existing code.

explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.

concerned with algorithms and the assignment of responsibilities between objects.



How to explain or describe a design pattern

- Context
- Problem
- Solution

Important: Design for Change

- Design patterns simplify future code changes
- However, if change is unlikely, their use may lead to overengineering

(1) Factory

we create object **without exposing** the creation logic to the client and refer to newly created object using a common interface.

Context: A system that uses communication channels

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```



Whenever there is the word **new**, it
is **glue** ⇒

The classes are glued together

Problem

- Applications may need to switch from TCP to UDP
- How can we design the system to support such changes and parameterize the communication protocol?

Solution: Factory Pattern

- Factory: a method that encapsulates object creation
- It provides a single point of change if the communication protocol must be updated (e.g., from TCP to UDP)

```
class ChannelFactory {  
    public static Channel create() { // static factory method  
        return new TCPChannel();  
    }  
}
```

Without a Factory

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

Without a Factory

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

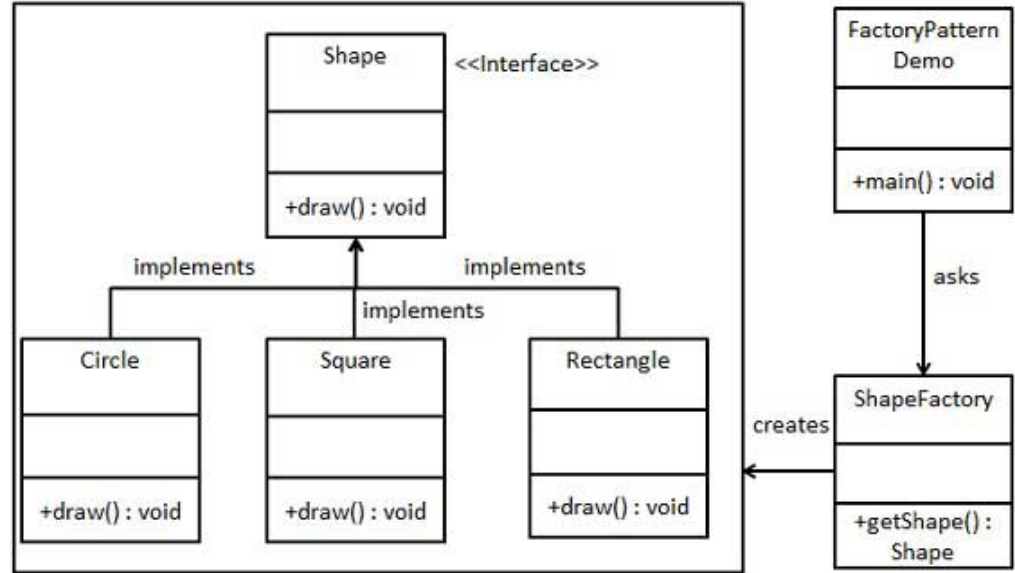


With a Factory

```
void f() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void g() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void h() {  
    Channel c = ChannelFactory.create();  
    ...  
}
```

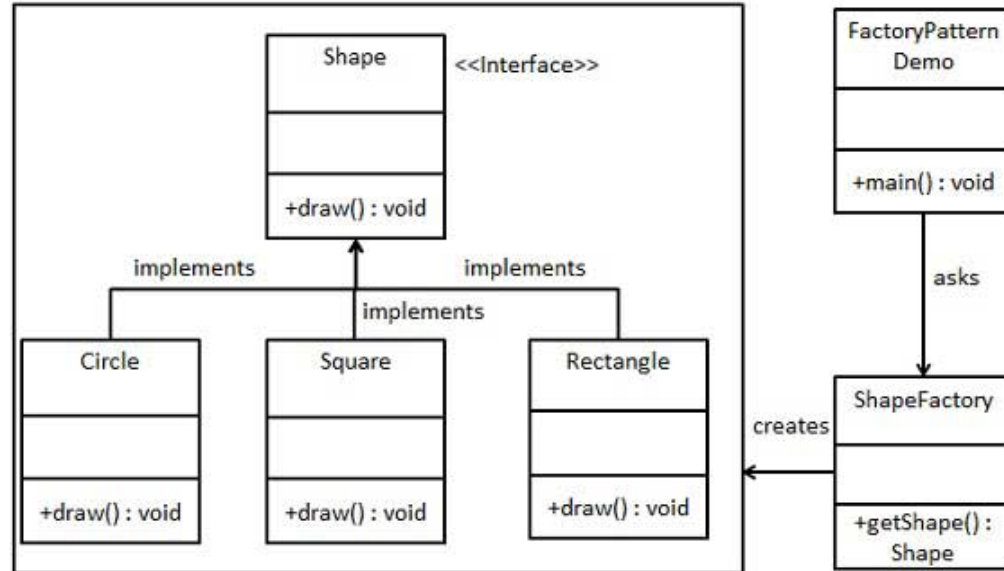
Factory : Shapes Example

```
public interface Shape {  
    void draw();  
}  
  
public class Rectangle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Rectangle::draw() method.");  
    }  
}  
  
public class Square implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Square::draw() method.");  
    }  
}
```



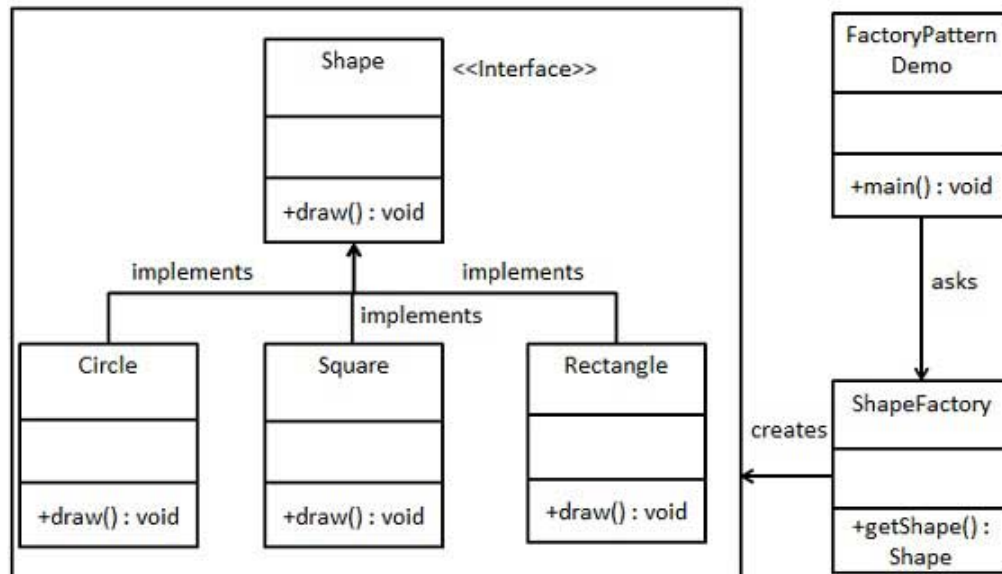
Factory : Shapes Example

```
public class ShapeFactory {  
  
    //use getShape method to get object of type shape  
    public Shape getShape(String shapeType){  
        if(shapeType == null){  
            return null;  
        }  
        if(shapeType.equalsIgnoreCase("CIRCLE")){  
            return new Circle();  
        }  
        else if(shapeType.equalsIgnoreCase("RECTANGLE")){  
            return new Rectangle();  
        }  
        else if(shapeType.equalsIgnoreCase("SQUARE")){  
            return new Square();  
        }  
        return null;  
    }  
}
```



Factory : Shapes Example

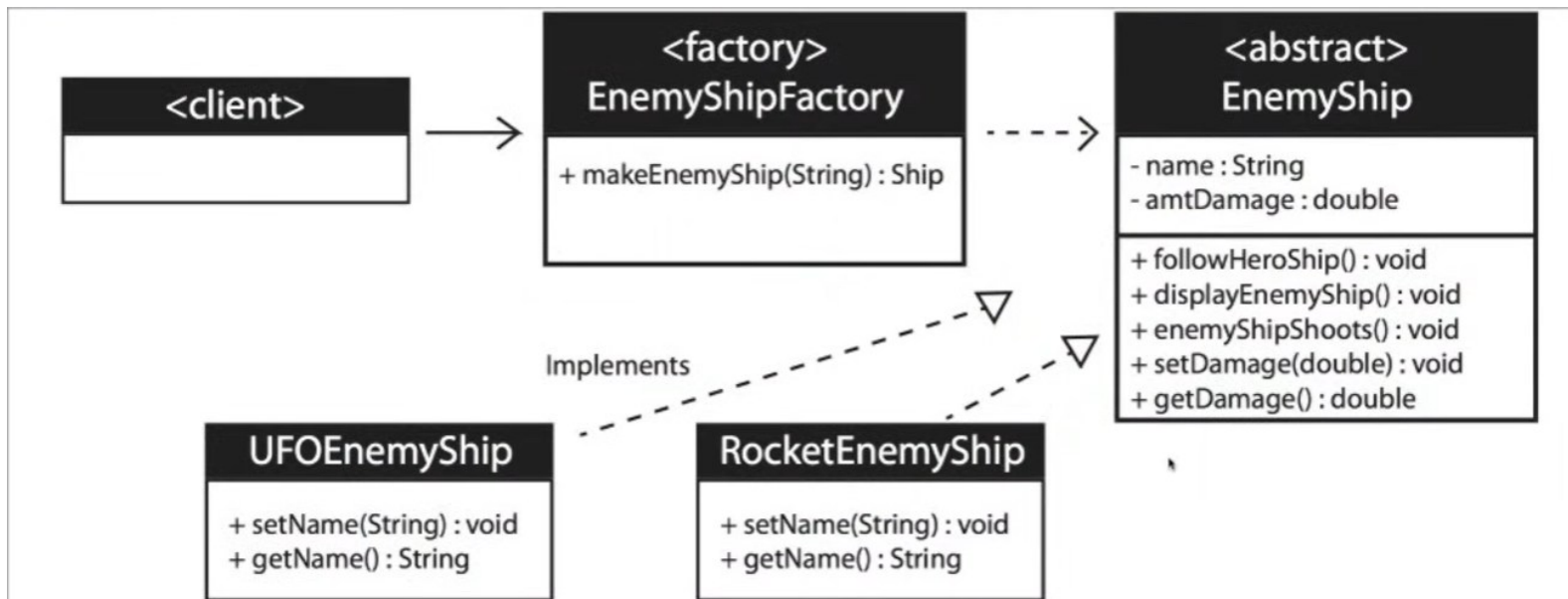
```
public class FactoryPatternDemo {  
  
    public static void main(String[] args) {  
        ShapeFactory shapeFactory = new ShapeFactory();  
  
        //get an object of Circle and call its draw method.  
        Shape shape1 = shapeFactory.getShape("CIRCLE");  
  
        //call draw method of Circle  
        shape1.draw();  
  
        //get an object of Rectangle and call its draw method.  
        Shape shape2 = shapeFactory.getShape("RECTANGLE");  
  
        //call draw method of Rectangle  
        shape2.draw();  
  
        //get an object of Square and call its draw method.  
        Shape shape3 = shapeFactory.getShape("SQUARE");  
  
        //call draw method of square  
        shape3.draw();  
    }  
}
```



Example taken from :

https://www.tutorialspoint.com/design_pattern/factory_pattern.htm

Factory: Another example



The **Factory Pattern** allows you to create objects without specifying the exact class of object that will be created.



EnemyShipTesting.java
<Store>



EnemyShipTesting.java
<Store>



```
public static void main(String[] args){

    // Create the factory object
    EnemyShipFactory shipFactory =
        new EnemyShipFactory();

    // Enemy ship object

    EnemyShip theEnemy = null;

    Scanner userInput = new Scanner(System.in);

    System.out.print("What type of ship? (U / R / B)");

    if (userInput.hasNextLine()){

        String typeOfShip = userInput.nextLine();

        theEnemy = shipFactory.makeEnemyShip(typeOfShip);

        if(theEnemy != null){

            doStuffEnemy(theEnemy);

        } else System.out.print("Please enter U, R,
            or B next time");

    }

}

}
```



UFOEnemyShip

```
public class UFOEnemyShip extends EnemyShip
{
    public UFOEnemyShip(){

        setName("UFO Enemy Ship");
        setDamage(20.0);
        setDamage(20.0);

    }

}
```

myShip

```
UFOEnemyShip extends EnemyShip
myShip(){
    UFO Enemy Ship";
    20.0);
    20.0);
}
```

When to Use the Factory Method

- When object creation logic is complex or varies based on conditions.
- When the system needs to be open for extension but closed for modification (OCP).
- When new product types may be added in the future.
- When you want to decouple client code from concrete implementations.

Real World Use Cases of Factory design pattern

- **Web Browsers (e.g., Chrome, Firefox)**

Browsers use factory methods to create different types of plugins or page renderers based on content type (e.g., PDF, HTML, Flash). This allows flexible and extensible handling of various media.

- **Payment Gateways (e.g., Stripe, PayPal)**

E-commerce platforms use factory methods to create different payment processors. Based on user selection, the factory returns an instance of the correct payment service without changing client logic.

- **Game Development (e.g., Unity, Unreal Engine)**

Games use factory methods to spawn enemies, items, or NPCs dynamically based on game level or environment. This decouples object creation from the game logic and enables easy scalability.

(2) Singleton

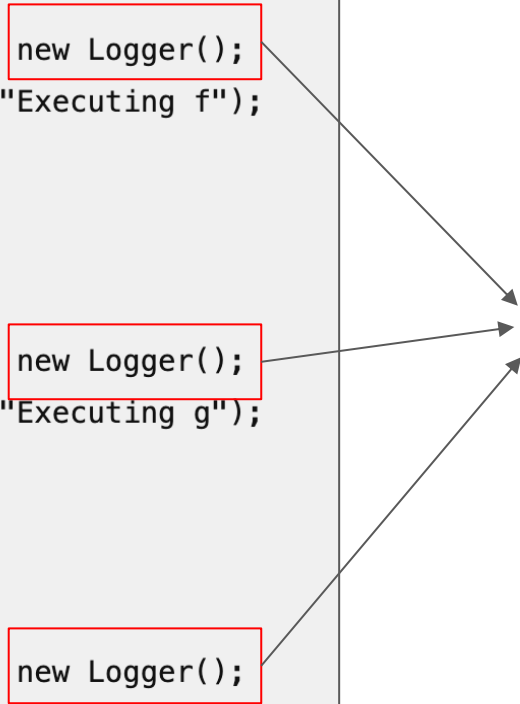
Singleton

- The Singleton Design Pattern ensures that a class has only one instance and provides a global access point to it

Context: A `Logger` class used across the system

```
void f() {  
    Logger log = new Logger();  
    log.println("Executing f");  
    ...  
}  
  
void g() {  
    Logger log = new Logger();  
    log.println("Executing g");  
    ...  
}  
  
void h() {  
    Logger log = new Logger();  
    log.println("Executing h");  
    ...  
}
```

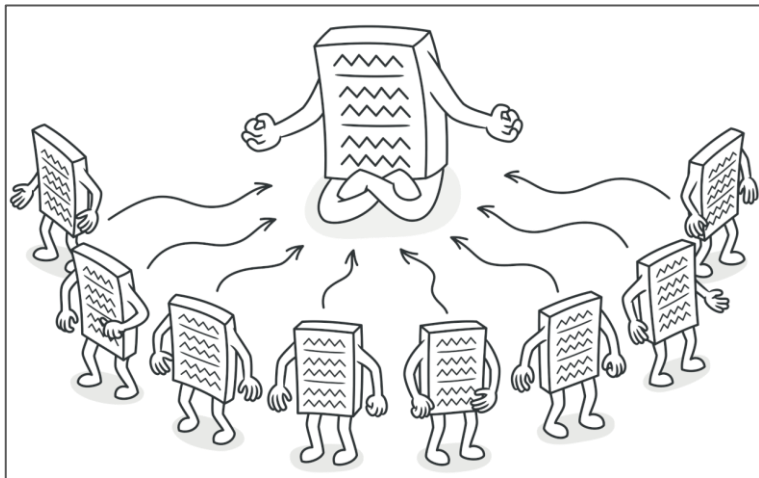
```
void f() {  
    Logger log = new Logger();  
    log.println("Executing f");  
    ...  
}  
  
void g() {  
    Logger log = new Logger();  
    log.println("Executing g");  
    ...  
}  
  
void h() {  
    Logger log = new Logger();  
    log.println("Executing h");  
    ...  
}
```



Problem: Each method creates
a separate `Logger` instance

Problem

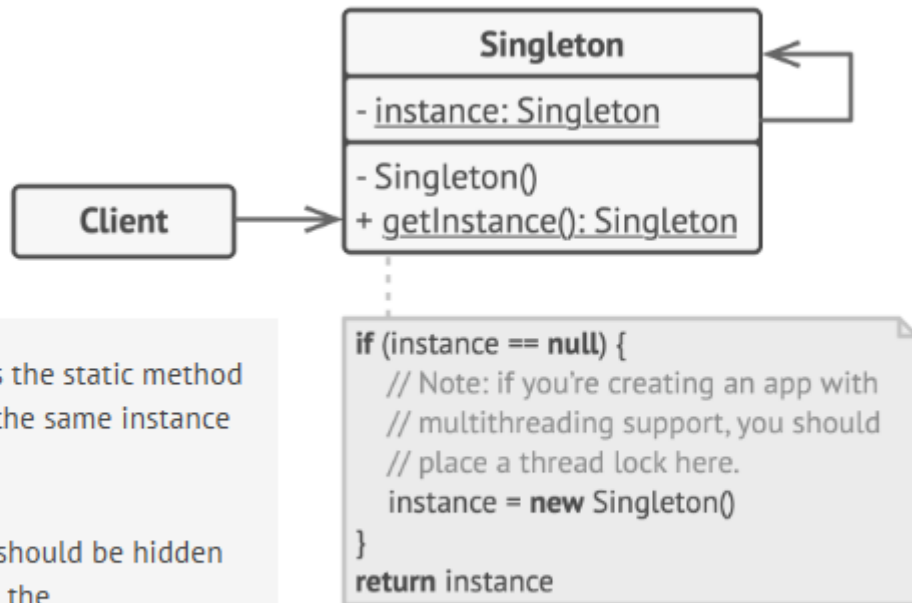
- All operations must be logged in the same file
- How can we ensure that all methods use the same `Logger` instance?



Solution: Singleton Pattern

- Convert the `Logger` class into a Singleton
- A Singleton is a class designed to ensure that at most one instance can exist

Singleton



1 The **Singleton** class declares the static method `getInstance` that returns the same instance of its own class.

The Singleton's constructor should be hidden from the client code. Calling the `getInstance` method should be the only way of getting the Singleton object.

```
class Logger {  
    private Logger() {} // prohibits new Logger() in clients  
  
    private static Logger instance; // single instance  
  
    public static Logger getInstance() {  
        if (instance == null) // 1st time getInstance is called  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // logs msg to console, but it could be to a file  
        System.out.println(msg);  
    }  
}
```

```
class Logger {  
  
    private Logger() {} // prohibits new Logger() in clients  
  
    private static Logger instance; // single instance  
  
    public static Logger getInstance() {  
        if (instance == null) // 1st time getInstance is called  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // logs msg to console, but it could be to a file  
        System.out.println(msg);  
    }  
}
```

```
class Logger {  
  
    private Logger() {} // prohibits new Logger() in clients  
  
    private static Logger instance; // single instance  
  
    public static Logger getInstance() {  
1      if (instance == null) // 1st time getInstance is called  
        instance = new Logger();  
2      return instance;  
    }  
  
    public void println(String msg) {  
        // logs msg to console, but it could be to a file  
        System.out.println(msg);  
    }  
}
```

```
class Logger {  
  
    private Logger() {} // prohibits new Logger() in clients  
  
    private static Logger instance; // single instance  
  
    public static Logger getInstance() {  
        if (instance == null) // 1st time getInstance is called  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // logs msg to console, but it could be to a file  
        System.out.println(msg);  
    }  
}
```

```
void f() {
```

```
    Logger log = Logger.getInstance();
```

```
    log.println("Executing f");
```

```
    ...
```

```
}
```

```
void g() {
```

```
    Logger log = Logger.getInstance();
```

```
    log.println("Executing g");
```

```
    ...
```

```
}
```

```
void h() {
```

```
    Logger log = Logger.getInstance();
```

```
    log.println("Executing h");
```

```
    ...
```

```
}
```

All methods share the same
Logger instance

Singleton: Example 2

```
class Application is
  method main() is
    Database foo = Database.getInstance()
    foo.query("SELECT ...")
    // ...
    Database bar = Database.getInstance()
    bar.query("SELECT ...")
    // The variable `bar` will contain the same object as
    // the variable `foo`.
```

```
// The Database class defines the `getInstance` method that lets
// clients access the same instance of a database connection
// throughout the program.
class Database is
  // The field for storing the singleton instance should be
  // declared static.
  private static field instance: Database

  // The singleton's constructor should always be private to
  // prevent direct construction calls with the `new`
  // operator.
  private constructor Database() is
    // Some initialization code, such as the actual
    // connection to a database server.
    // ...

  // The static method that controls access to the singleton
  // instance.
  public static method getInstance() is
    if (Database.instance == null) then
      acquireThreadLock() and then
        // Ensure that the instance hasn't yet been
        // initialized by another thread while this one
        // has been waiting for the lock's release.
        if (Database.instance == null) then
          Database.instance = new Database()
      return Database.instance

  // Finally, any singleton should define some business logic
  // which can be executed on its instance.
  public method query(sql) is
    // For instance, all database queries of an app go
    // through this method. Therefore, you can place
    // throttling or caching logic here.
    // ...
```

Real-World Applications of the Singleton Pattern

- **Logging Systems** : Maintain a consistent logging mechanism across an application.
- **Configuration Managers** : Centralize access to configuration settings.
- **Database Connections** : Manage a single point of database access.
- **Thread Pools** : Efficiently manage a pool of threads for concurrent tasks.
- **Cache Managers, Print Spoolers (Single Printer Queue) and Runtime Environments** (`java.lang.Runtime` is a singleton)

(3) Proxy

Proxy

- **Proxy** is a structural design pattern that lets you provide a substitute or placeholder for another object. A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.

Context: A book search function in a library system.

```
class BookSearch {  
    ...  
    Book getBook(String ISBN) { ... }  
    ...  
}
```

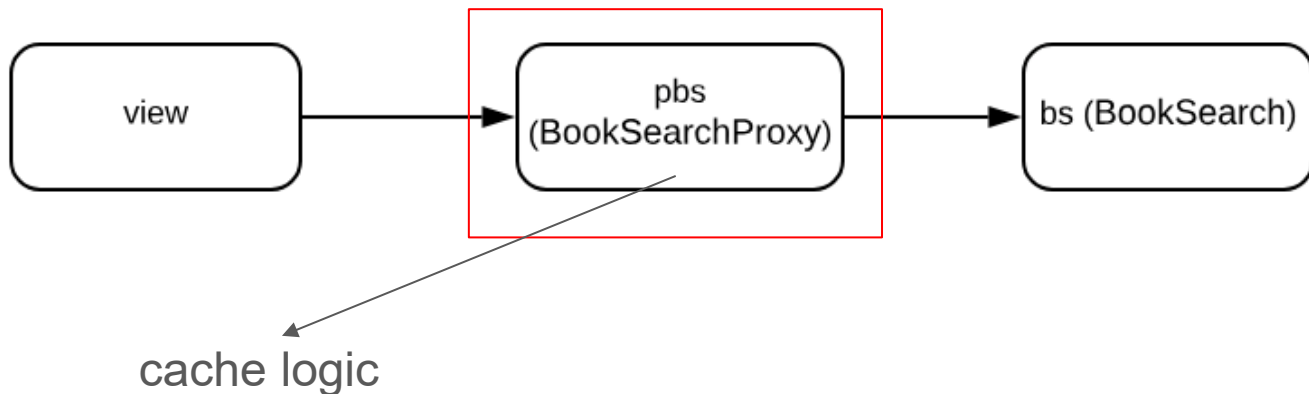
```
void main() {  
    BookSearch bs = new BookSearch();  
    ...  
    View view = new View(bs);  
    ...  
}
```

Problem: Using a cache to improve performance

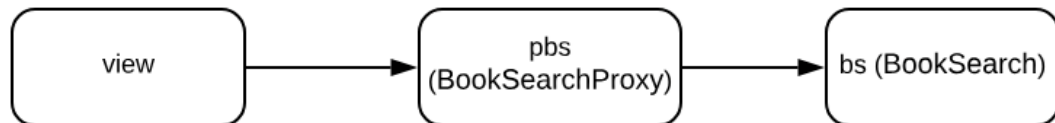
- To improve performance, we want to use a cache
- If the book is in the cache, return it immediately; otherwise, perform the search
- However, we want to avoid modifying the `BookSearch` class because it is already well tested and maintained by another developer

Solution: Proxy Pattern

- The Proxy pattern introduces an intermediary object between the client and the base object
- Clients interact with the proxy, which forwards requests to the base object and may add extra logic (e.g., caching)



```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```



```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs = new BookSearchProxy(bs);  
}
```

```
class BookSearchProxy implements BookSearchInterface {
```

```
    private BookSearchInterface base;
```

```
    BookSearchProxy (BookSearchInterface base) {  
        this.base = base;  
    }
```

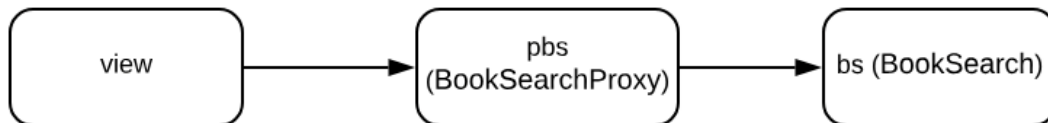
```
    Book getBook(String ISBN) {  
        if("book with ISBN in cache")  
            return "book from cache"  
        else {  
            Book book = base.getBook(ISBN);  
            if (book != null)  
                "add book to cache"  
            return book;  
        }  
    }  
    ...  
}
```

pbs
BookSearchProxy

bs (BookSearch)



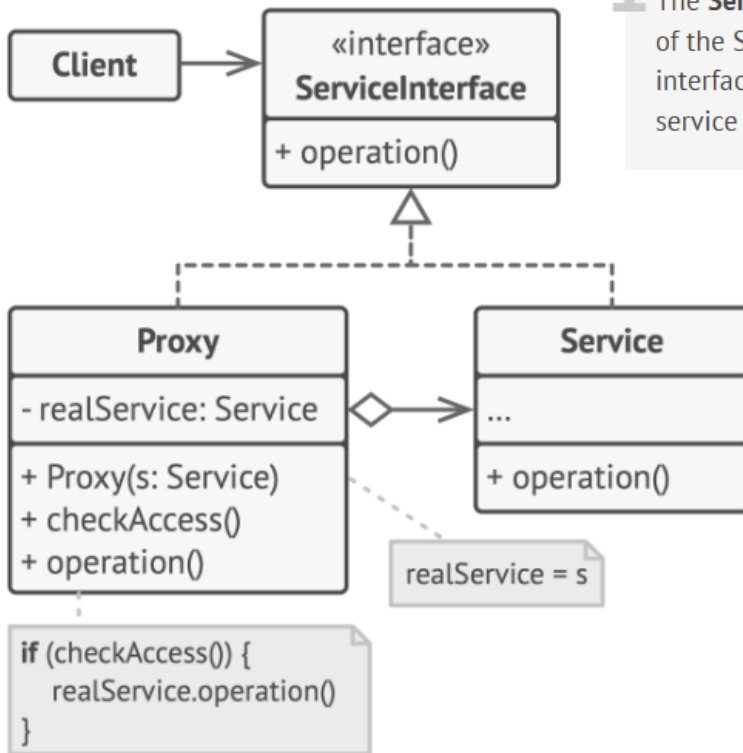
```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```



4 The **Client** should work with both services and proxies via the same interface. This way you can pass a proxy into any code that expects a service object.

3 The **Proxy** class has a reference field that points to a service object. After the proxy finishes its processing (e.g., lazy initialization, logging, access control, caching, etc.), it passes the request to the service object.

Usually, proxies manage the full lifecycle of their service objects.



1 The **Service Interface** declares the interface of the Service. The proxy must follow this interface to be able to disguise itself as a service object.

2 The **Service** is a class that provides some useful business logic.

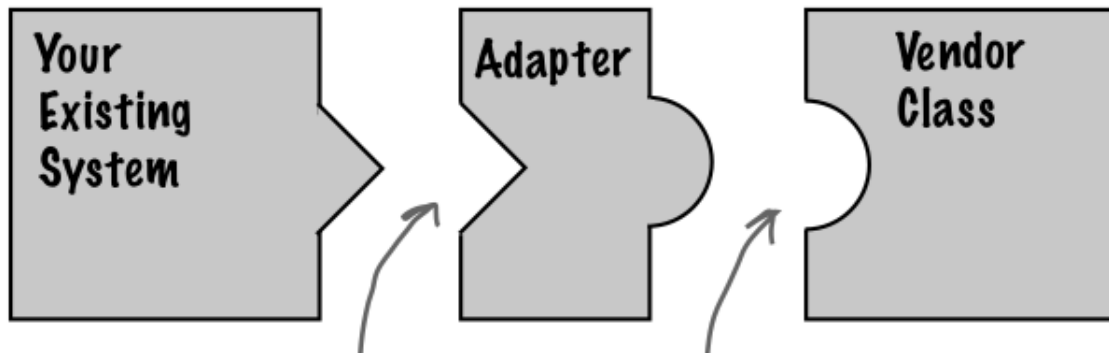
Proxy: a second example

```
abstract class Image {  
    // Client code  
}  
  
class ProxyImage extends Image {  
    // Image will be loaded from disk only when display() is called  
    image.display();  
  
    // Image will not be loaded again, as it has been cached in the Proxy  
    image.display();  
}  
}
```

(4) Adapter

Adapter

- Structural design pattern that allows objects with incompatible interfaces to collaborate
- The adapter acts as the middleman by receiving requests from the client and converting them into requests that make sense on the vendor classes.



Context: Multimedia Projectors Control System

```
class SamsungProjector {  
    public void start() { ... }  
    ...  
  
class LGProjector {  
    public void enable(int timer) { ... }  
    ...  
}
```

Because these drivers are provided by projector manufacturers, we cannot modify them

Problem

- The system uses a single `Projector` interface to abstract vendor-specific implementations

```
interface Projector {  
    void turnOn();  
}  
  
class ProjectorControlSystem {  
  
    void init(Projector projector) {  
        projector.turnOn(); // turns on any projector  
    }  
  
}
```

Problem

- However, vendor-provided drivers cannot be modified to implement the `Projector` interface

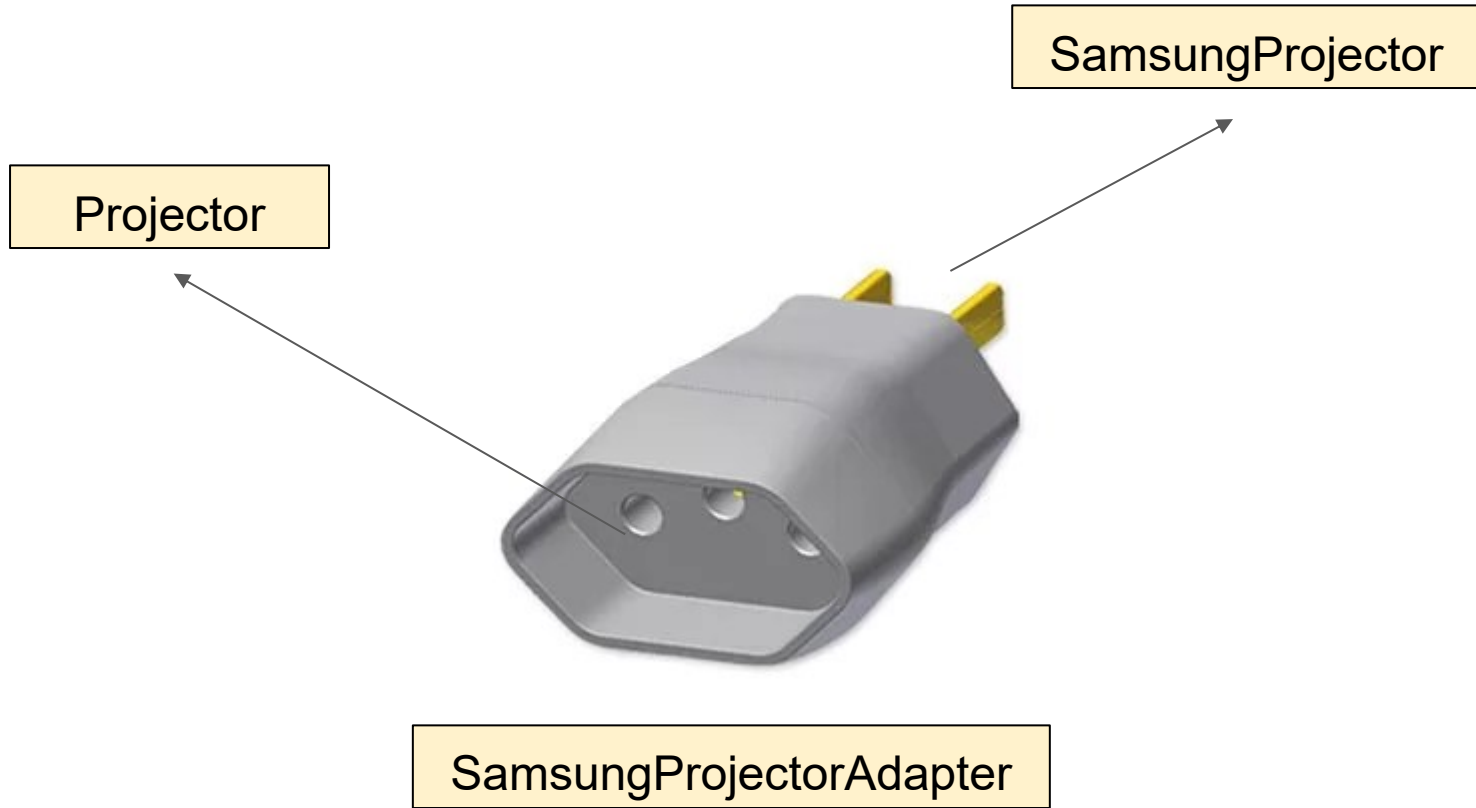
```
class SamsungProjector {  
    public void start() { ... }  
    ...  
  
class LGProjector {  
    public void enable(int timer) { ... }  
    ...  
}
```

Input interface

Output interface



Adapter Class

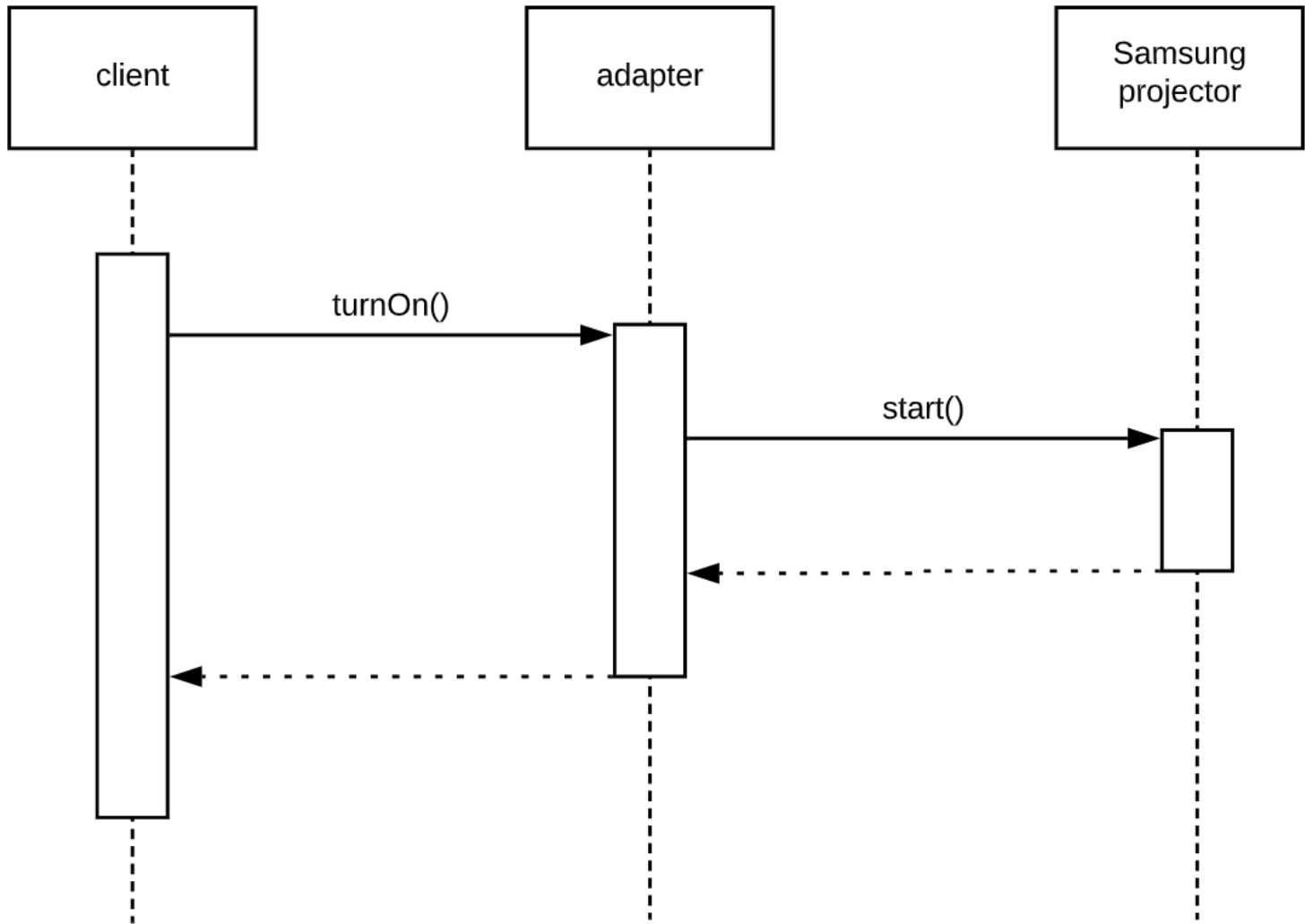


Solution: Adapter class

```
class SamsungProjectorAdapter implements Projector {  
  
    private SamsungProjector projector;  
  
    SamsungProjectorAdapter (SamsungProjector projector) {  
        this.projector = projector;  
    }  
  
    public void turnOn() {  
        projector.start();  
    }  
}
```

Solution: Adapter class

```
class SamsungProjectorAdapter implements Projector {  
    private SamsungProjector projector;  
  
    SamsungProjectorAdapter (SamsungProjector projector) {  
        this.projector = projector;  
    }  
  
    public void turnOn() {  
        projector.start();  
    }  
}
```

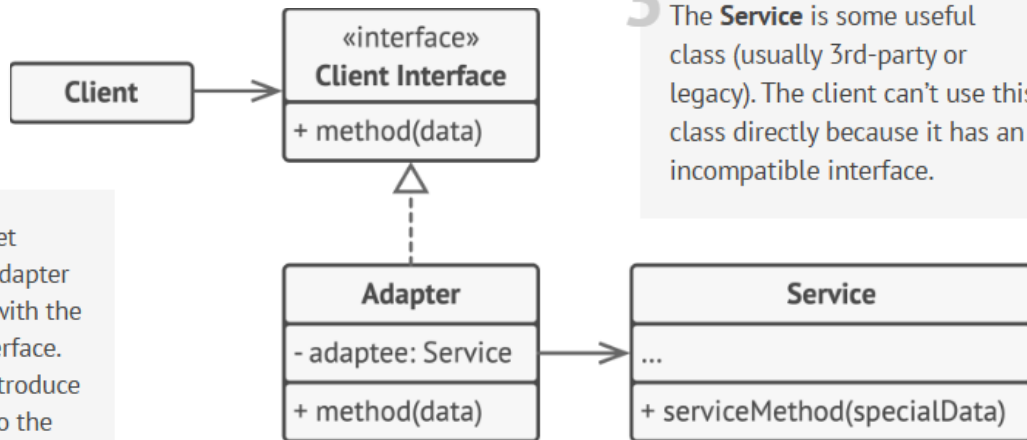


1 The **Client** is a class that contains the existing business logic of the program.

2 The **Client Interface** describes a protocol that other classes must follow to be able to collaborate with the client code.

3 The **Service** is some useful class (usually 3rd-party or legacy). The client can't use this class directly because it has an incompatible interface.

5 The client code doesn't get coupled to the concrete adapter class as long as it works with the adapter via the client interface. Thanks to this, you can introduce new types of adapters into the program without breaking the existing client code. This can be useful when the interface of the service class gets changed or replaced: you can just create a new adapter class without changing the client code.



```
specialData = convertToServiceFormat(data)
return adaptee.serviceMethod(specialData)
```

4 The **Adapter** is a class that's able to work with both the client and the service: it implements the client interface, while wrapping the service object. The adapter receives calls from the client via the client interface and translates them into calls to the wrapped service object in a format it can understand.

Uses Of Adapter Design Pattern

We can use adapter design pattern when:

- Enables communication between incompatible systems.
- Reuses existing code or libraries without rewriting.
- Simplifies integration of new components, keeping the system flexible.

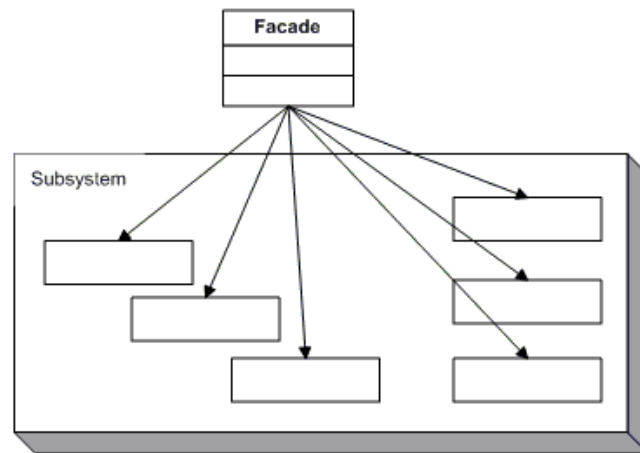
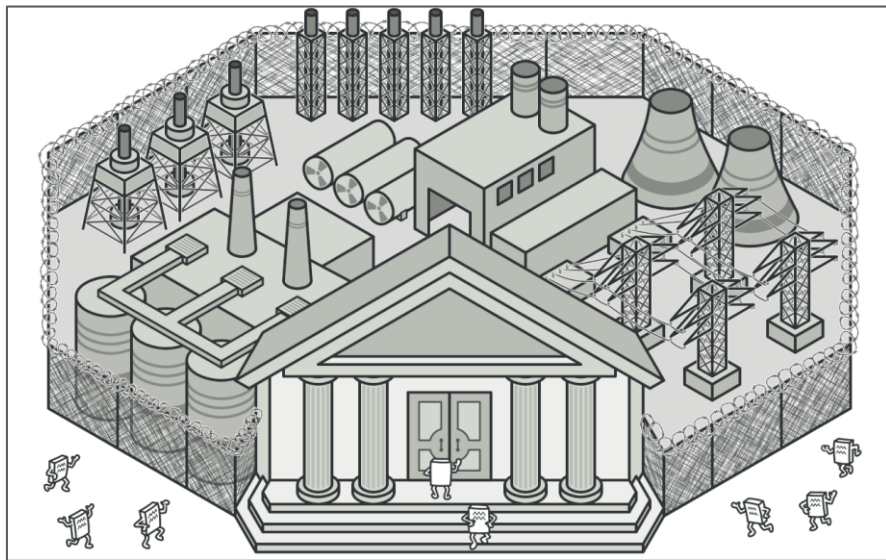
(5) Facade

Facade design pattern

- It simplifies interactions with complex subsystems. It provides a unified, easy-to-use interface while hiding internal details, reducing complexity for clients and promoting cleaner, more maintainable code.

Context, Problem & Solution

- Module M is used by multiple other modules (context)
- But M's interface is complex, and clients find it difficult to use (problem)
- Thus, we should provide a simpler interface for M, known as a Facade (solution)



<https://refactoring.guru/design-patterns/facade>

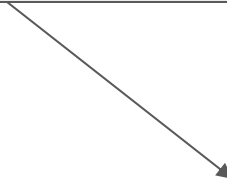
Example: Interpreter

```
Scanner s = new Scanner("prog1.abc");  
Parser p = new Parser(s);  
AST ast = p.parse();  
CodeGenerator code = new CodeGenerator(ast);  
code.eval();
```

```
Scanner s = new Scanner("prog1.abc");  
Parser p = new Parser(s);  
AST ast = p.parse();  
CodeGenerator code = new CodeGenerator(ast);  
code.eval();
```

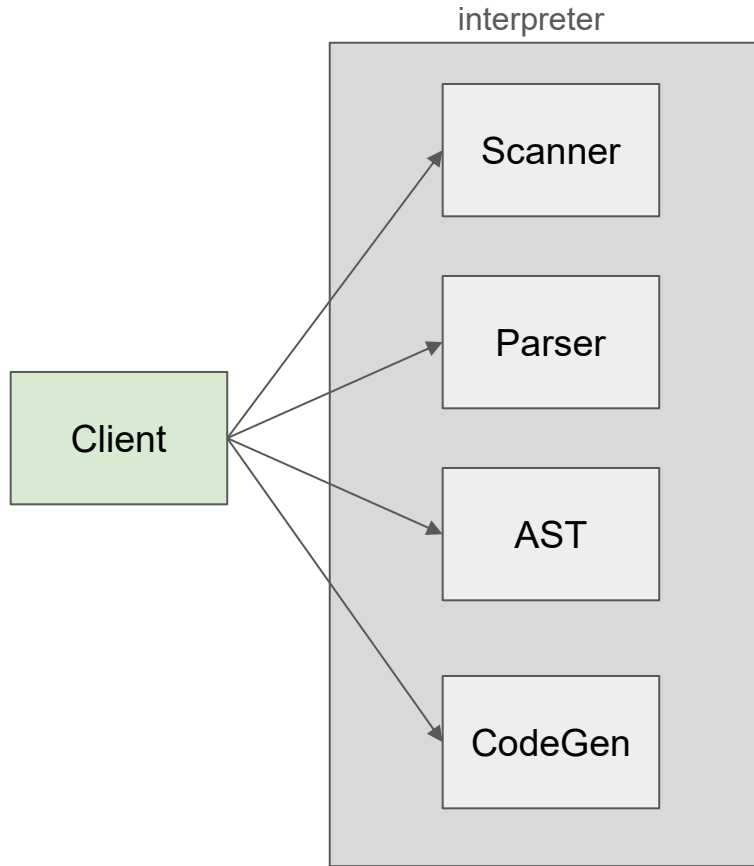


```
new Interpreter("prog1.abc").eval();
```

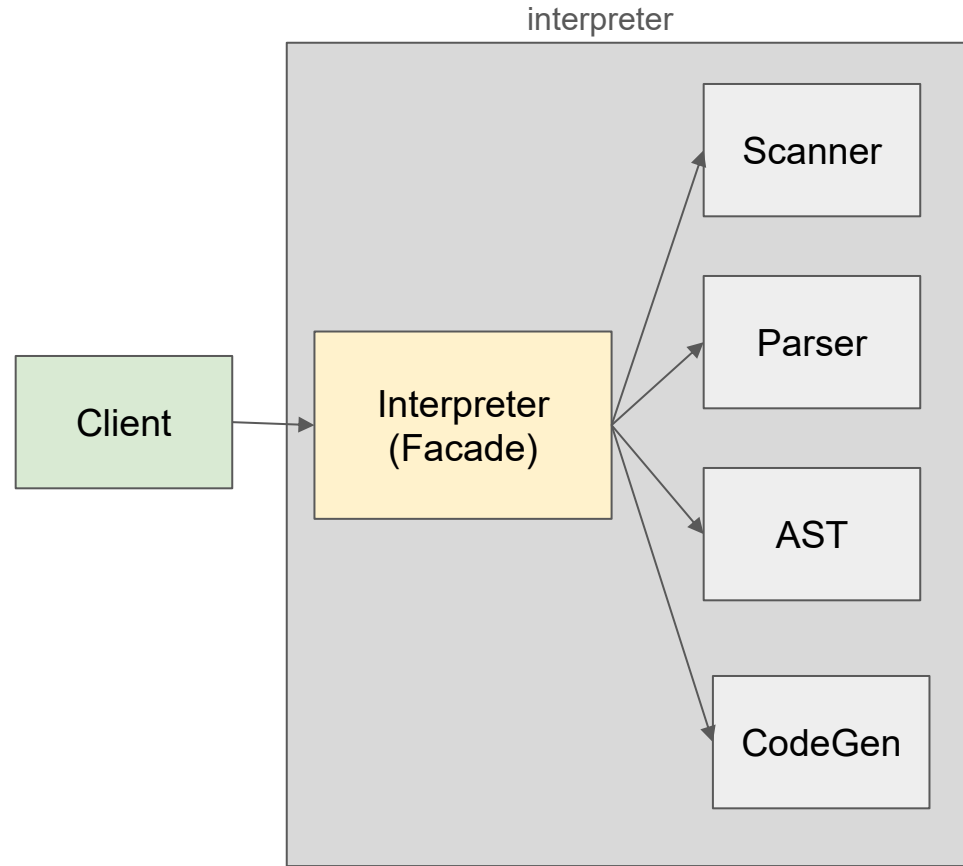


A Facade provides a simplified interface to a complex subsystem

Without a Facade



With a Facade



(6) Decorator

Context: A system that uses communication channels

```
interface Channel {  
    void send(String msg);  
    String receive();  
}  
  
class TCPChannel implements Channel {  
    ...  
}  
  
class UDPChannel implements Channel {  
    ...  
}
```

The same example introduced
for the Factory pattern

Problem

- Basic channels (TCP, UDP) are not sufficient
- We need to add extra functionalities, such as:
 - Data compression/decompression
 - Buffering
 - Logging
 - Other enhancements

Problem

- **Problem:**
- 4 features × 2 protocols = 32+ subclasses needed
- Exponential growth with more features
- Rigid, inflexible combinations
- Violates Open/Closed Principle

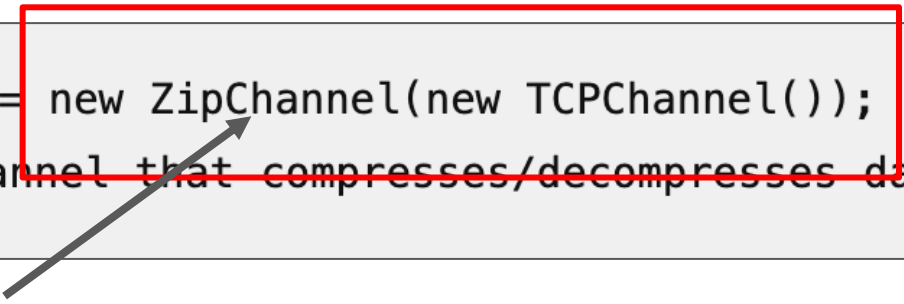
- `TCPZipChannel` extends `TCPChannel`
- `TCPBufferedChannel` extends `TCPChannel`
- `TCPBufferedZipChannel` extends `TCPZipChannel` extends `TCPChannel`
- `TCPLogChannel` extends `TCPChannel`
- `TCPLogBufferedZipChannel` extends `TCPBufferedZipChannel` extends `TCPZipChannel` extends `TCPChannel`
- `UDPZipChannel` extends `UDPChannel`
- `UDPBufferedChannel` extends `UDPChannel`
- `UDPBufferedZipChannel` extends `UDPZipChannel` extends `UDPChannel`
- `UDPLogChannel` extends `UDPChannel`
- `UDPLogBufferedZipChannel` extends `UDPBufferedZipChannel` extends `UDPZipChannel` extends `UDPChannel`

Solution: Decorator Pattern

- The Decorator pattern allows adding functionality to a class through composition rather than inheritance
- This avoids creating an excessive number of subclasses

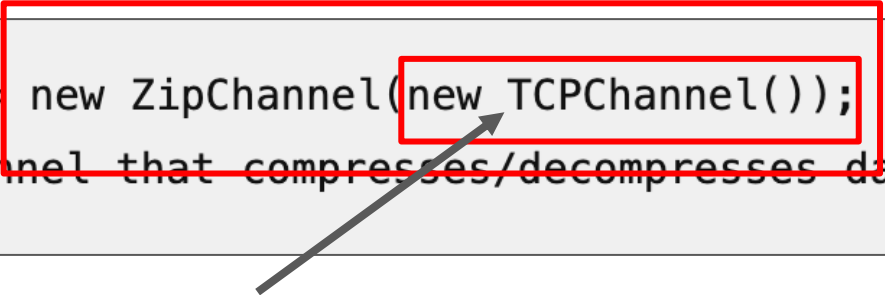
Example

```
channel = new ZipChannel(new TCPChannel());  
// TCPChannel that compresses/decompresses data
```



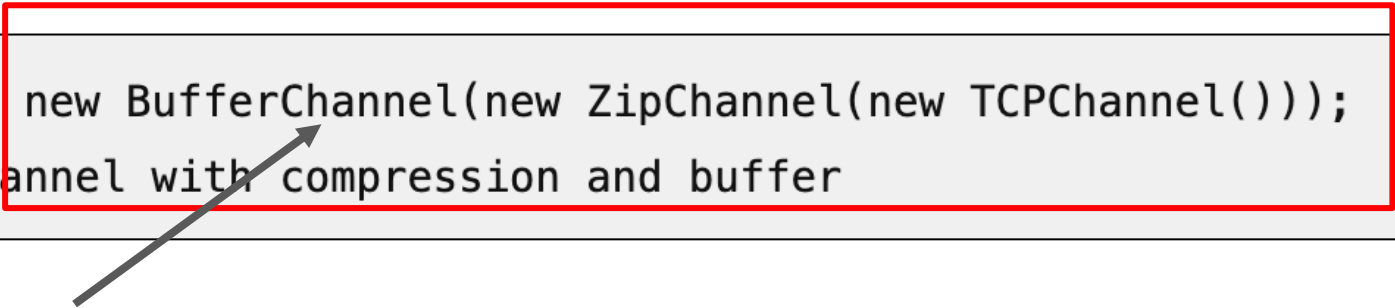
Example

```
channel = new ZipChannel(new TCPChannel());  
// TCPChannel that compresses/decompresses data
```



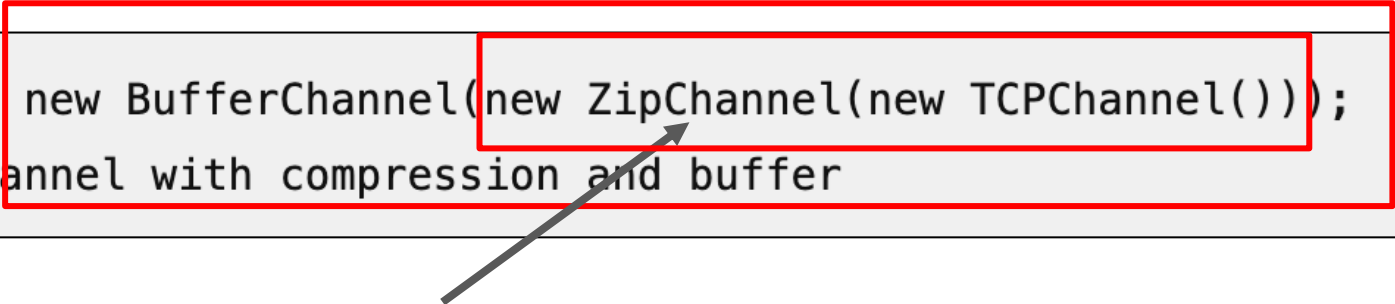
Another example

```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



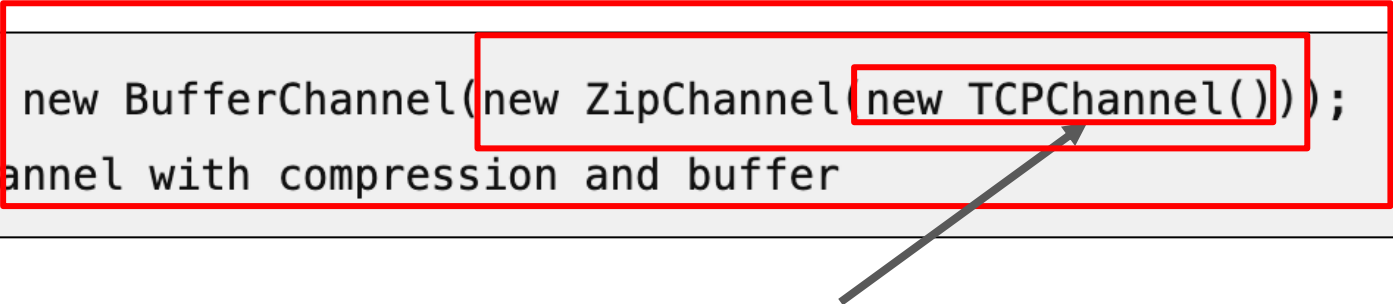
Another example

```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



Another example

```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



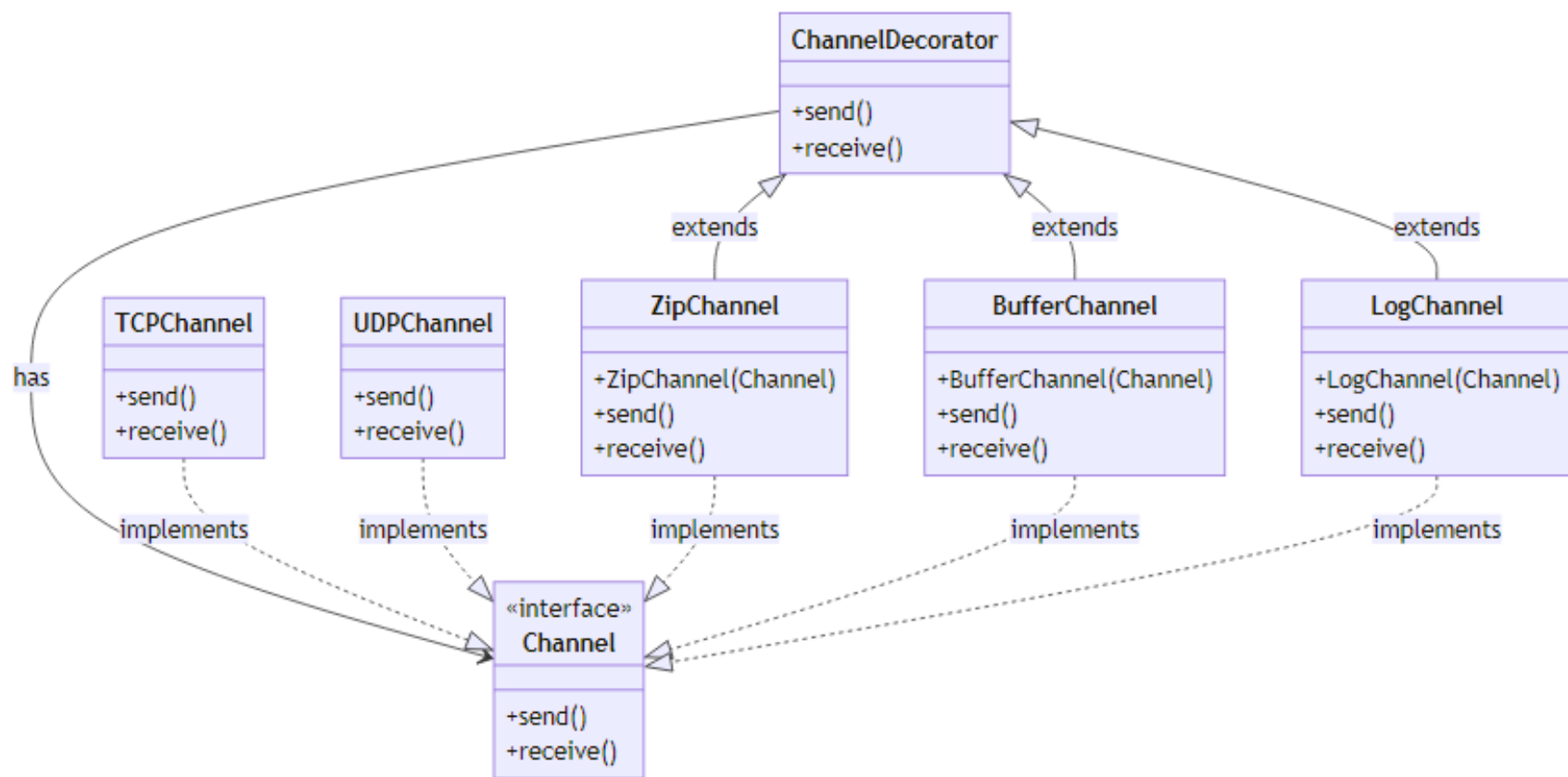
The diagram illustrates the nested construction of the channel. A large red rectangle encloses the entire code snippet. Inside it, a smaller red rectangle highlights the argument to the BufferChannel constructor, which is the ZipChannel. Within the ZipChannel's argument, another red rectangle highlights the TCPChannel constructor call. A grey arrow points from the bottom right towards the TCPChannel constructor, indicating the flow of the nested construction.

```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



Like nested gift boxes, Decorators wrap around objects (e.g., `TCPChannel` or `UDPChannel`) to add new features

Decorator Implementation



ChannelDecorator

All decorators extend this class

```
class ChannelDecorator implements Channel {  
  
    private Channel channel;  
  
    public ChannelDecorator(Channel channel) {  
        this.channel = channel;  
    }  
  
    public void send(String msg) {  
        channel.send(msg);    // Delegate to wrapped channel  
    }  
  
    public String receive() {  
        return channel.receive();    // Delegate to wrapped channel  
    }  
  
}
```

ZipChannel Implementation

```
class ZipChannel extends ChannelDecorator {
```

```
    public ZipChannel(Channel c) {  
        super(c);  
    }
```

```
    public void send(String msg) {  
        "compress message msg"  
        super.send(msg);    // Delegate after adding behavior  
    }
```

```
    public String receive() {  
        String msg = super.receive();  
        "decompress message msg"  
        return msg;  
    }
```

```
}
```

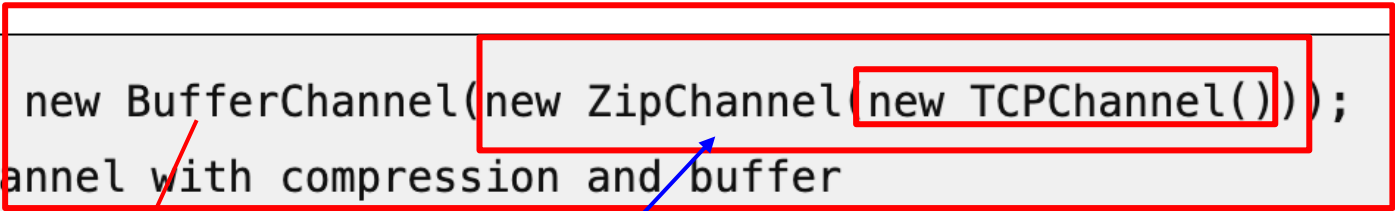
```
Channel c = new ZipChannel(new TCPChannel());  
c.send("Hello, world");
```

The `send` call in the last line triggers the following method executions:

- First, `ZipChannel.send` is executed, which compresses the message.
- After compression, `ZipChannel.send` invokes `super.send`, which, in turn, executes `ChannelDecorator.send`.
- `ChannelDecorator.send` delegates the call to its internal `Channel`, which, in this example, is a `TCPChannel`.
- Finally, we reach `TCPChannel.send`, which transmits the compressed message via TCP.

Example

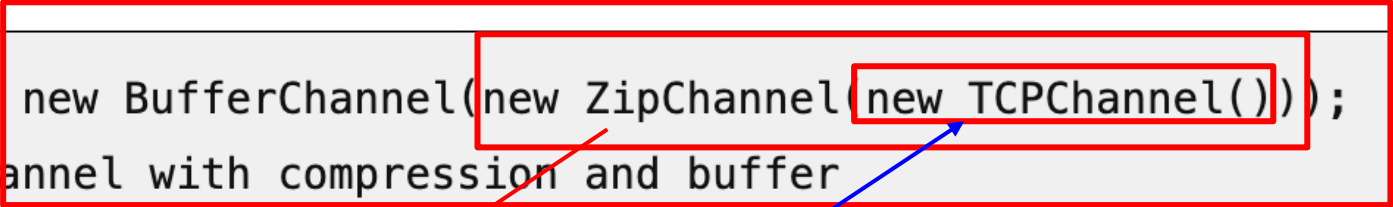
```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



```
class BufferChannel extends ChannelDecorator {  
    ... super.channel  
}
```

Another example

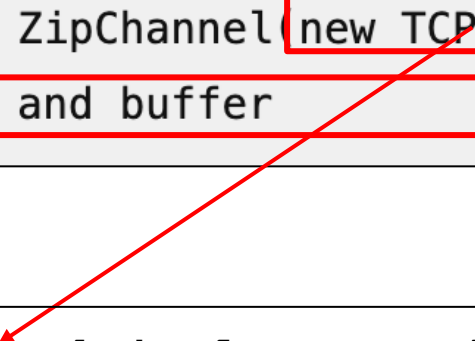
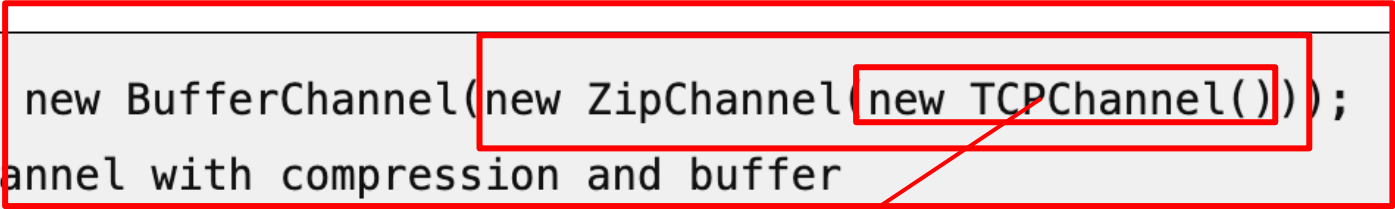
```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



```
class ZipChannel extends ChannelDecorator {  
    ... super.channel  
}
```

Another example

```
channel= new BufferChannel(new ZipChannel(new TCPChannel()));  
// TCPChannel with compression and buffer
```



```
class TCPChannel implements Channel {  
    // final Channel instance  
}
```

(7) Strategy

Startegy

- a behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.

Context: Data Structures Library

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    public void sort() {  
        ... // sorts the list using Quicksort  
    }  
}
```

Current State:

- Sorting algorithm hardcoded to Quicksort
- No flexibility in algorithm choice
- Clients have different performance needs

Problem

- The `MyList` class is closed for extension
- For example, it cannot easily support other sorting algorithms (e.g., ShellSort, HeapSort)

Solution: Strategy Pattern

- The Strategy pattern allows parameterizing the algorithms used by a class
- Thus, it enables a class to support new algorithms (for example, different sorting algorithms)

Step #1: Define a Strategy hierarchy, where each strategy corresponds to a specific algorithm

```
abstract class SortStrategy {  
    abstract void sort(MyList list);  
}  
  
class QuickSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}  
  
class ShellSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}
```

Step #2: Modify the `MyList` class to use a Strategy

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
}
```

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
}
```

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
}
```

100

```
MyList list = new MyList();
```

```
list.add(items...); // Use default Quicksort
```

```
list.sort();
```

```
// Switch to Mergesort
```

```
list.setSortStrategy(new MergeSortStrategy());
```

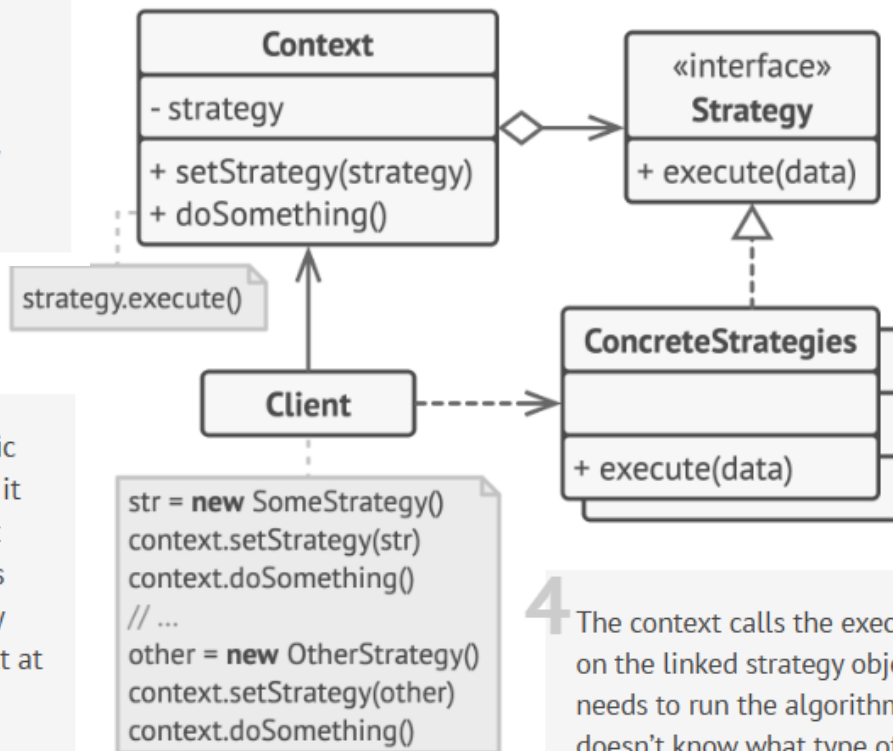
```
list.sort();
```

```
// Switch to Heapsort for different data
```

```
list.setSortStrategy(new HeapSortStrategy());
```

```
list.sort();
```

1 The **Context** maintains a reference to one of the concrete strategies and communicates with this object only via the strategy interface.



2 The **Strategy** interface is common to all concrete strategies. It declares a method the context uses to execute a strategy.

3 **Concrete Strategies** implement different variations of an algorithm the context uses.

5 The **Client** creates a specific strategy object and passes it to the context. The context exposes a setter which lets clients replace the strategy associated with the context at runtime.

4 The context calls the execution method on the linked strategy object each time it needs to run the algorithm. The context doesn't know what type of strategy it works with or how the algorithm is executed.

(8) Observer

Observer

- Defines a one-to-many dependency between objects where when one object (subject) changes state, all its dependents (observers) are notified and updated automatically.

- **Social Media Notifications**

Users (observers) get updates when someone they follow (subject) posts new content or status.

- **Stock Market Apps**

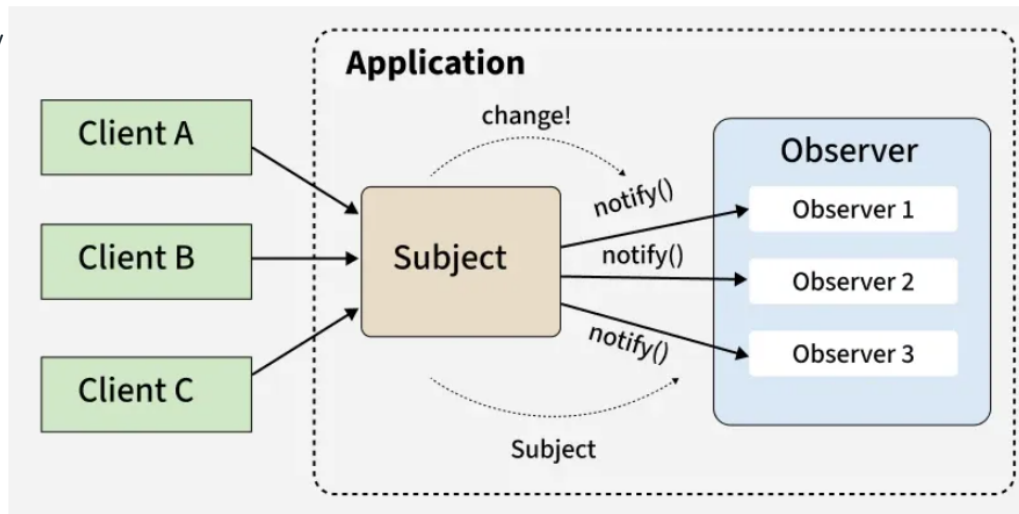
Investors receive real-time updates when stock prices (subjects) change.

- **Event Listeners in GUIs**

UI components observe user actions like clicks or keyboard input and respond accordingly.

- **Weather Monitoring Systems**

Multiple displays or devices update automatically when the central weather data changes.



Context: Weather Station System

- **Main classes:** `Temperature` **and** `Thermometer`
- Multiple thermometer types: digital, analog, web, mobile, etc
- When the temperature changes, all thermometers must be updated

Problem

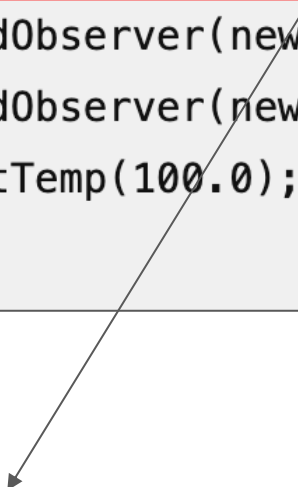
- We want to avoid coupling `Temperature` directly to `Thermometer` classes
- Reasons:
 - Keep the domain class independent from UI classes
 - Allow new thermometer types to be added easily

Solution: Observer Pattern

- Implements a one-to-many relationship:
 - **Subject** (`Temperature`)
 - **Observers** (`Thermometer`)
- When the subject changes, it notifies all observers.
- But the subject remains unaware of the observers' concrete types

Example

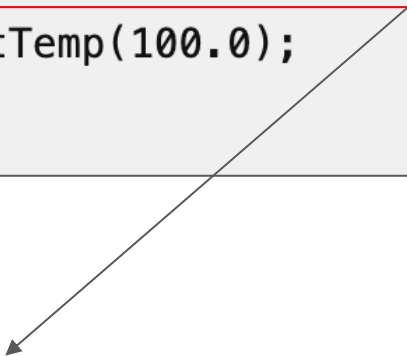
```
void main() {  
    Temperature t = new Temperature();  
    t.addObserver(new CelsiusThermometer());  
    t.addObserver(new FahrenheitThermometer());  
    t.setTemp(100.0);  
}
```



Subject

Example

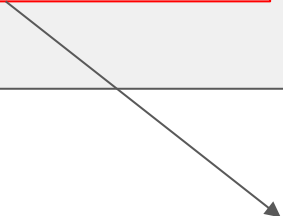
```
void main() {  
    Temperature t = new Temperature();  
    t.addObserver(new CelsiusThermometer());  
    t.addObserver(new FahrenheitThermometer());  
    t.setTemp(100.0);  
}
```



Two observers

Example

```
void main() {  
    Temperature t = new Temperature();  
    t.addObserver(new CelsiusThermometer());  
    t.addObserver(new FahrenheitThermometer());  
    t.setTemp(100.0);  
}
```



Notifies all observers

```
abstract class Subject {  
    private List<Observer> observers = new ArrayList<>();  
  
    public void addObserver(Observer observer) {  
        observers.add(observer);  
    }  
  
    public void notifyObservers() {  
        for (Observer obs : observers) {  
            obs.update(this);  
        }  
    }  
}
```

```
interface Observer {  
    public void update(Subject s);  
}
```

```
class Temperature extends Subject {  
  
    private double temp;  
  
    public double getTemp() {  
        return temp;  
    }  
  
    public void setTemp(double temp) {  
        this.temp = temp;  
        notifyObservers();  
    }  
}
```

```
class Temperature extends Subject {
```

```
    private double temp;
```

```
    public double getTemp() {  
        return temp;  
    }
```

```
    public void setTemp(double temp) {  
        this.temp = temp;  
        notifyObservers();  
    }  
}
```

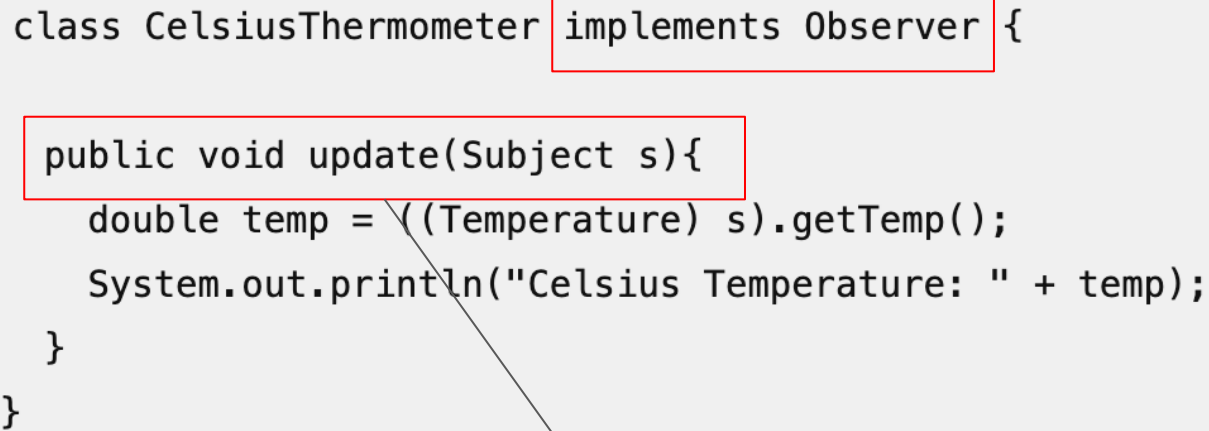
Implements addObserver and
notifyObservers

```
class Temperature extends Subject {  
  
    private double temp;  
  
    public double getTemp() {  
        return temp;  
    }  
  
    public void setTemp(double temp) {  
        this.temp = temp;  
        notifyObservers();  
    }  
}
```

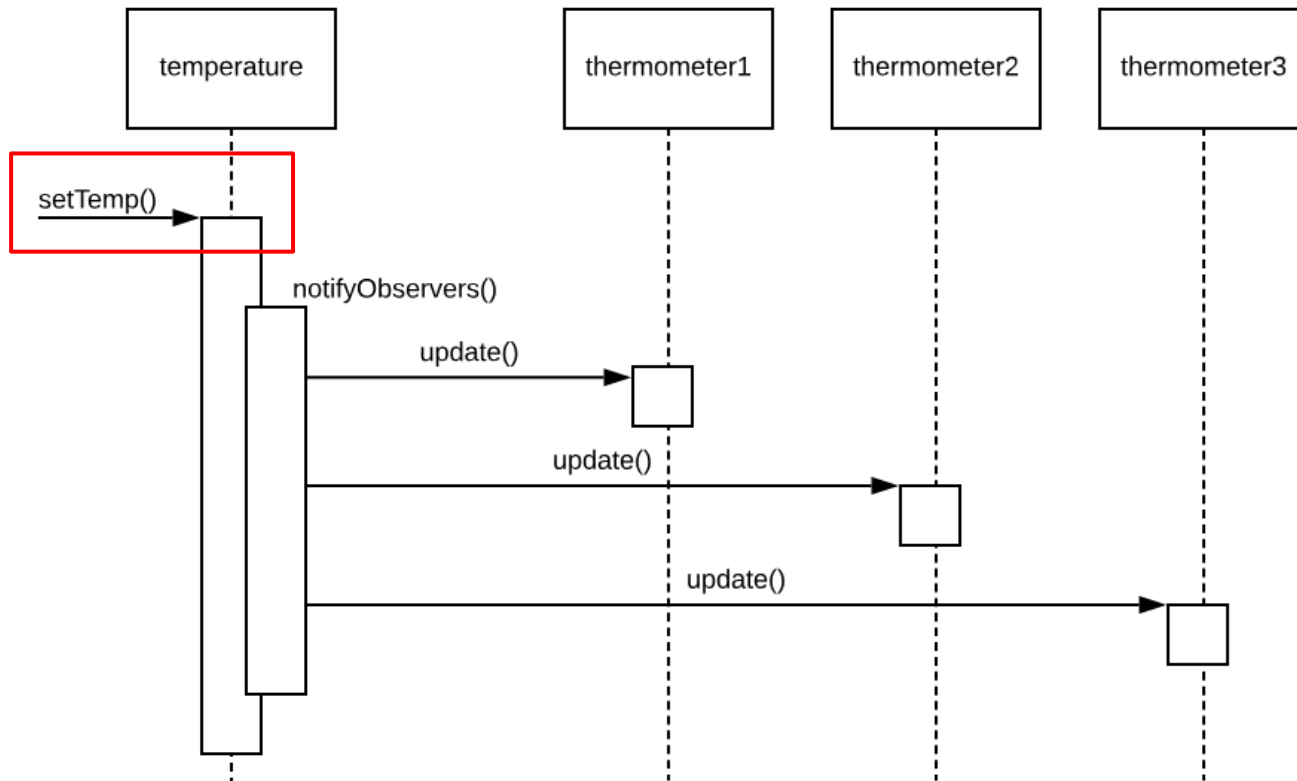


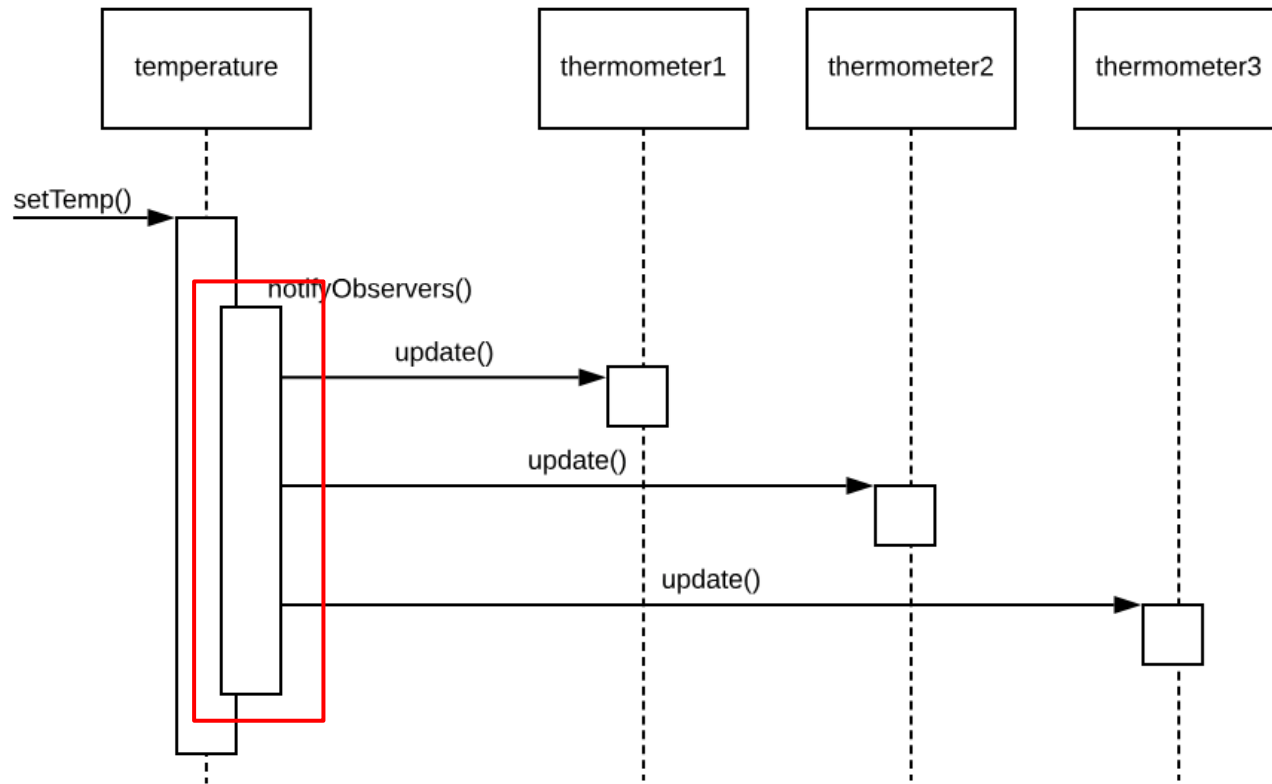
When the temperature changes, the subject notifies all registered observers

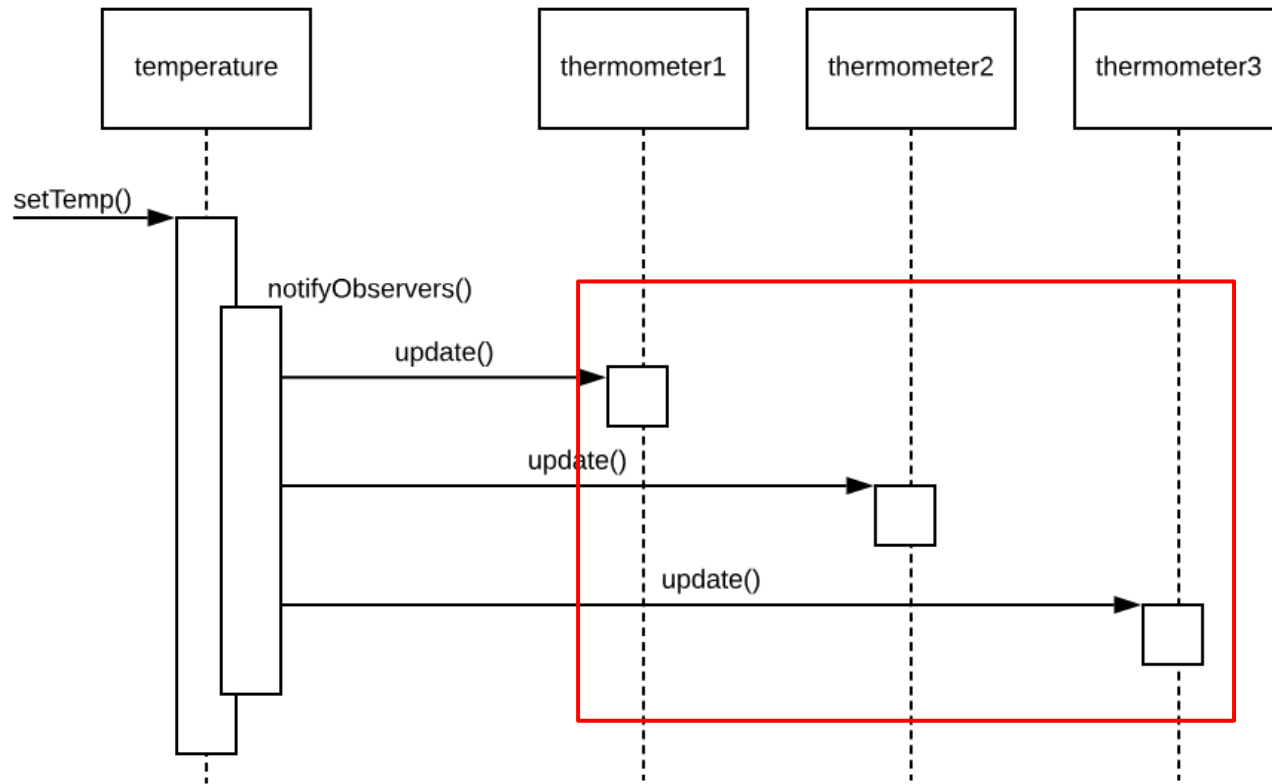
```
class CelsiusThermometer implements Observer {  
  
    public void update(Subject s){  
        double temp = ((Temperature) s).getTemp();  
        System.out.println("Celsius Temperature: " + temp);  
    }  
}
```



All observers must implement the `update()` method.
When `Temperature.notifyObservers()` is called,
it executes `update()` in each observer.

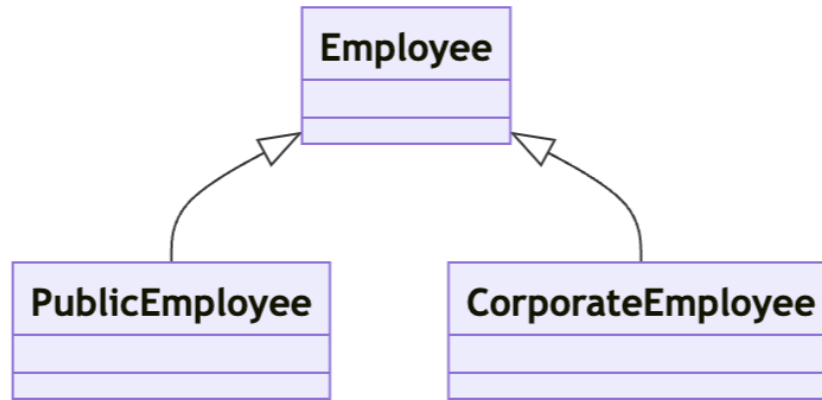






(9) Template Method

Context: A payroll system for different employee types



Problem

- Problem: Salary calculation follows similar steps for public and corporate employees, but with different details
- Goal: Define the overall workflow in the parent class (`Employee`)
- Subclasses implement the specific details of each step

Solution: Template Method Pattern

- The Template Method pattern defines the skeleton of an algorithm in a base class, while leaving certain steps (abstract methods) to subclasses
- Subclasses customize specific steps but preserve the overall algorithm structure

```
abstract class Employee {  
  
    double salary;  
    ...  
    abstract double calcRetirementDeductions();  
    abstract double calcHealthPlanDeductions();  
    abstract double calcOtherDeductions();  
  
    public double calcNetSalary() { // template method  
        double retirement = calcRetirementDeductions();  
        double health = calcHealthPlanDeductions();  
        double other = calcOtherDeductions();  
        return salary - retirement - health - other;  
    }  
}
```

```
abstract class Employee {
```

```
    double salary;
```

```
    ...
```

```
    abstract double calcRetirementDeductions();
```

```
    abstract double calcHealthPlanDeductions();
```

```
    abstract double calcOtherDeductions();
```

```
    public double calcNetSalary() { // template method
```

```
        double retirement = calcRetirementDeductions();
```

```
        double health = calcHealthPlanDeductions();
```

```
        double other = calcOtherDeductions();
```

```
        return salary - retirement - health - other;
```

```
    }
```

```
}
```

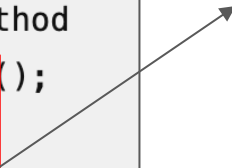
Implemented by the
subclasses



```
abstract class Employee {  
  
    double salary;  
    ...  
    abstract double calcRetirementDeductions();  
    abstract double calcHealthPlanDeductions();  
    abstract double calcOtherDeductions();  
  
    public double calcNetSalary() { // template method  
        double retirement = calcRetirementDeductions();  
        double health = calcHealthPlanDeductions();  
        double other = calcOtherDeductions();  
        return salary - retirement - health - other;  
    }  
}
```

```
abstract class Employee {  
  
    double salary;  
    ...  
    abstract double calcRetirementDeductions();  
    abstract double calcHealthPlanDeductions();  
    abstract double calcOtherDeductions();  
  
    public double calcNetSalary() { // template method  
        double retirement = calcRetirementDeductions();  
        double health = calcHealthPlanDeductions();  
        double other = calcOtherDeductions();  
        return salary - retirement - health - other;  
    }  
}
```

Template method for
calculating net salary



When should we avoid design patterns?

When should we avoid design patterns?

- Design patterns are tools for enabling changes
- If a system is unlikely to change, applying design patterns is unnecessary

Example: Strategy

Trade-off: Design for Change vs Complexity

- Using design patterns increases complexity in order to support future changes
- For instance, they often require creating additional classes

Implementation without a design pattern

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    public void sort() {  
        ... // sorts the list using Quicksort  
    }  
}
```

1 class

Implementation without a design pattern

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    public void sort() {  
        ... // sorts the list using Quicksort  
    }  
}
```

1 class

1 class (with more code)
+1 abstract class
+2 classes with strategies

Implementation using a design pattern

```
class MyList {  
  
    ... // list data  
    ... // list methods: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
}
```

```
abstract class SortStrategy {  
    abstract void sort(MyList list);  
}  
  
class QuickSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}  
  
class ShellSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}
```

Patternitis: A possible example of overusing patterns, adding unnecessary complexity

org.springframework.aop.framework

Class AbstractSingletonProxyFactoryBean

[java.lang.Object](#)

└ [org.springframework.aop.framework.ProxyConfig](#)

└ [org.springframework.aop.framework.AbstractSingletonProxyFactoryBean](#)

All Implemented Interfaces:

[Serializable](#), [BeanClassLoaderAware](#), [FactoryBean](#), [InitializingBean](#)

Direct Known Subclasses:

[TransactionProxyFactoryBean](#)

```
public abstract class AbstractSingletonProxyFactoryBean
extends ProxyConfig
implements FactoryBean, BeanClassLoaderAware, InitializingBean
```

Convenient proxy factory bean superclass for proxy factory beans that create only singletons.

Manages pre- and post-interceptors (references, rather than interceptor names, as in [ProxyFactoryBean](#)) and provides consistent interface management.

End